

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>Н2</u>	<u>Программная инженерия и интеллектуальные системы</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Проектирование архитектуры программных средств</u>	
	наименование дисциплины	

УЧЕБНО–ПРАКТИЧЕСКАЯ РАБОТА

1

номер задания (при наличии)

РАЗРАБОТКА СПЕЦИФИКАЦИИ ТРЕБОВАНИЙ

при наличии указать тему учебно–практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О729Б

Барышников О.А.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

Зими́на Д.В.

подпись

фамилия и инициалы

Оценка / балльная оценка _____

дата проверки

г. Санкт–Петербург
2026 г.

СОДЕРЖАНИЕ

1. Предварительные замечания к проекту	3
1.1. Цели и рамки проекта	3
1.2. Деловой контекст	3
1.3. Участники проекта	4
1.4. Идеи в отношении решений	4
1.5. Обзор документа	4
2. Системные сервисы	4
2.1. Рамки системы	4
2.2. Функциональные требования	5
2.3. Требования к данным	5
3. Системные ограничения	6
3.1. Требования к интерфейсу	6
3.2. Требования к производительности	6
3.3. Требования к безопасности	6
3.4. Эксплуатационные требования	6
3.5. Политические и юридические требования	7
3.6. Другие ограничения	7
4. Проектные вопросы	7
4.1. Открытые вопросы	7
4.2. Предварительный план–график	7
4.3. Предварительный бюджет	7
5. Приложения	8
5.1. Глоссарий	8
5.2. Деловые документы и формы	8
5.3. Ссылки	8

1. Предварительные замечания к проекту

1.1. Цели и рамки проекта

Целью проекта является разработка программного инструмента для автоматизированной трансформации монолитных веб-приложений (написанных на языке Python) в микросервисную архитектуру с последующей контейнеризацией.

Модуль должен обеспечивать:

- эвристический анализ исходного кода и выделение независимых микросервисов;
- автоматическую очистку моделей баз данных (устранение перекрестных связей);
- генерацию прокси-заглушек (Stub) для предотвращения циклических импортов;
- создание конфигурационных файлов инфраструктуры;
- управление жизненным циклом сгенерированных контейнеров через встроенный графический интерфейс.

Проект охватывает разработку архитектуры, программного ядра, графического интерфейса пользователя и подготовку модуля к защите в рамках ВКР.

1.2. Деловой контекст

Процесс миграции устаревших монолитных систем на микросервисную архитектуру требует огромных временных затрат со стороны разработчиков и DevOps-инженеров. Ручное разделение кода часто приводит к нарушению зависимостей, циклическим импортам и возникновению состояний гонки при инициализации баз данных.

Разрабатываемый программный инструмент предназначен для автоматизации данного процесса.

Система должна автоматически:

- сканировать монолит и строить карту зависимостей;
- генерировать изолированный код для каждого сервиса;
- Настроить единую точку для скрывания внутренней топологии сети.

Основной эффект внедрения:

- кардинальное снижение трудозатрат на рефакторинг монолитных систем;
- исключение человеческого фактора при написании конфигураций Docker;
- обеспечение безопасного и изолированного развертывания сервисов.

1.3. Участники проекта

Заказчик – IT-компания или команда разработчиков, планирующая миграцию монолита.

Руководитель проекта – научный руководитель ВКР.

Backend-разработчик – студент.

DevOps-инженер / Архитектор – студент.

Разработчик алгоритмов – студент.

Тестировщик – разработчик совместно с научным руководителем.

1.4. Идеи в отношении решений

Программный инструмент должен быть реализован:

- на языке программирования Python 3;
- с использованием библиотеки `customtkinter` для построения современного графического интерфейса;
- с использованием шаблонизатора `Jinja2` для динамической генерации файлов конфигурации;
- с интеграцией подсистемы `Docker` для локального оркестрирования контейнеров через системные вызовы.

1.5. Обзор документа

В разделе «Системные сервисы» описываются функции системы и требования к данным. В разделе «Системные ограничения» приводятся ограничения по интерфейсу, производительности, безопасности и эксплуатации. В разделе «Проектные вопросы» отражаются открытые вопросы, предварительный план–график и бюджет.

2. Системные сервисы

2.1. Рамки системы

Система включает:

- графический интерфейс пользователя с панелью управления;
- ядро эвристического сканирования исходного кода (`scanner.py`);
- ядро трансформации и генерации артефактов (`generator.py`);
- встроенный эмулятор терминала для отображения логов.

Инструмент получает на вход директорию с исходным кодом монолита,

выполняет лексический и структурный анализ, генерирует модифицированный код сервисов и артефакты развертывания, после чего позволяет запустить полученную микросервисную сеть в Docker.

2.2. Функциональные требования

Программный модуль должен обеспечивать:

- выбор директории с исходным кодом через диалоговое окно файловой системы ОС.
- определение типа модулей («service» или «shared lib») на основе эвристического поиска файлов маршрутизации (например, routes.py).
- автоматическое удаление жестких связей db.ForeignKey из моделей SQLAlchemy.
- генерацию динамических Stub-объектов для недостающих зависимостей.
- создание конфигурации Nginx (API Gateway) для маршрутизации HTTP-запросов.
- генерацию единого файла docker-compose.yaml с настроенными проверками healthcheck для СУБД PostgreSQL.
- запуск ресурсоемких процессов генерации в изолированном потоке (threading) для предотвращения зависания GUI.
- передачу команд управления (docker compose up / down) в консоль ОС.
- перехват потока вывода (stdout) и эвристическую раскраску логов (успех, ошибки, системные сообщения) во встроенном текстовом терминале.

2.3. Требования к данным

Система должна обрабатывать и хранить:

- входные данные:
 - исходный код монолитного Python-приложения;
 - файлы зависимостей.
- выходные данные:
 - изолированные директории для каждого микросервиса (скопированный и модифицированный исходный код);
 - сгенерированные файлы Dockerfile, entrypoint.py / run.py;
 - конфигурационный файл nginx.conf;

- файл оркестрации `docker-compose.yaml`.
- журнальные данные:
 - сообщения о ходе генерации;
 - сообщения об ошибках;
 - логи процессов сборки и запуска Docker-контейнеров.

Хранение персональных данных не предусмотрено.

3. Системные ограничения

3.1. Требования к интерфейсу

Приложение должно предоставлять современный оконный графический интерфейс.

Предусмотрены:

- элементы управления должны включать поля ввода путей;
- управляющие кнопки с цветовой индикацией;
- текстовый терминал должен использовать моноширинный шрифт.

Взаимодействие с вычислительным центром осуществляется через HTTPS.

3.2. Требования к производительности

Сканирование и генерация артефактов для монолита среднего размера (до 50 модулей) должны занимать не более 5-10 секунд.

Графический интерфейс должен оставаться отзывчивым (non-blocking) на протяжении всего процесса генерации и сборки образов.

Инициализация базы данных и запуск микросервисов должны строго контролироваться механизмом healthcheck во избежание состояний гонки.

3.3. Требования к безопасности

Процесс анализа и трансформации кода должен выполняться исключительно локально на машине пользователя.

Сгенерированная сеть микросервисов (app-network) должна быть закрытой, доступ к сервисам извне должен осуществляться исключительно через проксирующий шлюз Nginx.

3.4. Эксплуатационные требования

Программа должна функционировать:

- в круглосуточном режиме (24/7);
- при изменяющихся погодных условиях;
- при переменной освещенности.

Поддерживаемая ОС – MacOS.

Минимальные требования к оборудованию:

- M1 CPU;
- от 8 ГБ ОЗУ;
- SSD от 256 ГБ.

3.5. Политические и юридические требования

Программный модуль не должен нарушать действующее законодательство Российской Федерации.

Передача данных должна соответствовать требованиям информационной безопасности организации–заказчика.

3.6. Другие ограничения

Разработка выполняется в рамках ВКР.

Сроки ограничены учебным графиком.

4. Проектные вопросы

4.1. Открытые вопросы

Планируется ли в будущем поддержка генерации конфигураций для других систем управления базами данных (например, MySQL или MongoDB)?

Требуется ли внедрение механизма отката изменений (rollback) в случае частичной ошибки генерации артефактов?

4.2. Предварительный план–график

01.02.2026 – 15.02.2026 – анализ предметной области.

15.02.2026 – 01.03.2026 – формирование требований и архитектуры.

01.03.2026 – 01.04.2026 – разработка ядра трансформации кода.

01.04.2026 – 01.05.2026 – реализация классификатора.

01.05.2026 – 20.05.2026 – интеграция и тестирование.

20.05.2026 – 01.06.2026 – подготовка отчетной документации

4.3. Предварительный бюджет

Проект выполняется в рамках выпускной квалификационной работы.

Дополнительные финансовые вложения не требуются, так как используется локальное вычислительное оборудование и программное обеспечение с открытым исходным кодом.

5. Приложения

5.1. Глоссарий

GPU – графический процессор.

API Gateway – архитектурный паттерн.

Состояние гонки – ошибка проектирования многопоточных или распределенных систем, при которой порядок выполнения неконтролируем.

ИС – информационная система.

5.2. Деловые документы и формы

нет

5.3. Ссылки

нет